

Uppsala University
Department of Information Technology

Assignment 3

Distributed Infrastructure, Model Serving and CI/CD

Course: Data Engineering II

Arnab Kumar Ghosh

May 6, 2026

Contents

1	Task 1: Single Server Deployment	2
1.1	Overview	2
1.2	Application Architecture	2
1.3	CloudInit Configuration	2
1.4	Starting the Production VM	3
1.5	Questions	3
2	Task 2: Single Server Deployment with Docker Containers	5
2.1	Overview	5
2.2	Docker Compose Architecture	5
2.3	CloudInit Configuration (with Docker)	6
2.4	Scaling the Workers	6
2.5	Questions	6
3	Task 3: Deployment of Multiple Servers Using Ansible	9
3.1	Overview	9
3.2	Infrastructure	9
3.3	Questions	9
4	Task 4: CI/CD Using Git Hooks	11
4.1	Overview	11
4.2	Architecture	11
4.3	The post-receive Hook	11
4.4	Running the Pipeline	12
4.5	Questions	12

1 Task 1: Single Server Deployment

1.1 Overview

In this task, I deployed a machine learning model serving application on a single virtual machine. CloudInit handled the dynamic contextualization. The application has three parts: a Flask web frontend, a Celery worker, and a RabbitMQ message broker. The model is a Keras/TensorFlow neural network trained on the Pima Indians Diabetes dataset.

I used the OpenStack Python API from a client VM to automate the deployment. CloudInit installed all software and started all services at first boot. I did not need to log in to the production server manually.

1.2 Application Architecture

The production server has three components:

- **Flask app** (`app.py`): runs the web frontend on port 5100. It receives prediction requests from the browser and sends them to Celery as tasks.
- **RabbitMQ**: sits between Flask and Celery. It holds tasks in a queue until a worker is ready.
- **Celery worker** (`workerA.py`): takes a task from the queue, loads the Keras model, runs the prediction, and sends the result back.

The data flow is:

Browser → Flask (`app.py`) → RabbitMQ → Celery worker (`workerA.py`) → Flask → Browser

1.3 CloudInit Configuration

The file `cloud-cfg.txt` is given to OpenStack when the production VM is created. CloudInit runs the steps below automatically on the first boot:

Listing 1: `cloud-cfg.txt` – single server without Docker

```
1 #cloud-config
2
3 apt_update: true
4 apt_upgrade: true
5 packages:
6   - python3-pip
7   - python3-dev
8   - build-essential
9   - rabbitmq-server
10
11 byobu_default: system
12
13 runcmd:
14   - pip3 install "celery" "tensorflow==2.10.0" "amqp" "flask==2.3.1" "future" "numpy
    <2.0"
15   - git clone https://github.com/sztoor/model_serving.git
16   - celery --workdir=/model_serving/single_server_without_docker/production_server \
    -A workerA worker --detach --loglevel=debug --concurrency=1 -n wkr1@backend
17   - python3 /model_serving/single_server_without_docker/production_server/app.py \
    --host=0.0.0.0 --port=5100 &
```

1.4 Starting the Production VM

On the client VM, I sourced the OpenStack RC file and installed the OpenStack Python libraries. Then I ran the following command:

Listing 2: Starting the production VM from the client

```
python3 start_instance.py
```

The script read credentials from the environment, created the VM with the CloudInit user-data, and waited until the instance status was ACTIVE. After I attached a floating IP, the application was reachable at:

- Welcome page: <http://130.238.27.96:5100>
- Predictions page: <http://130.238.27.96:5100/predictions>

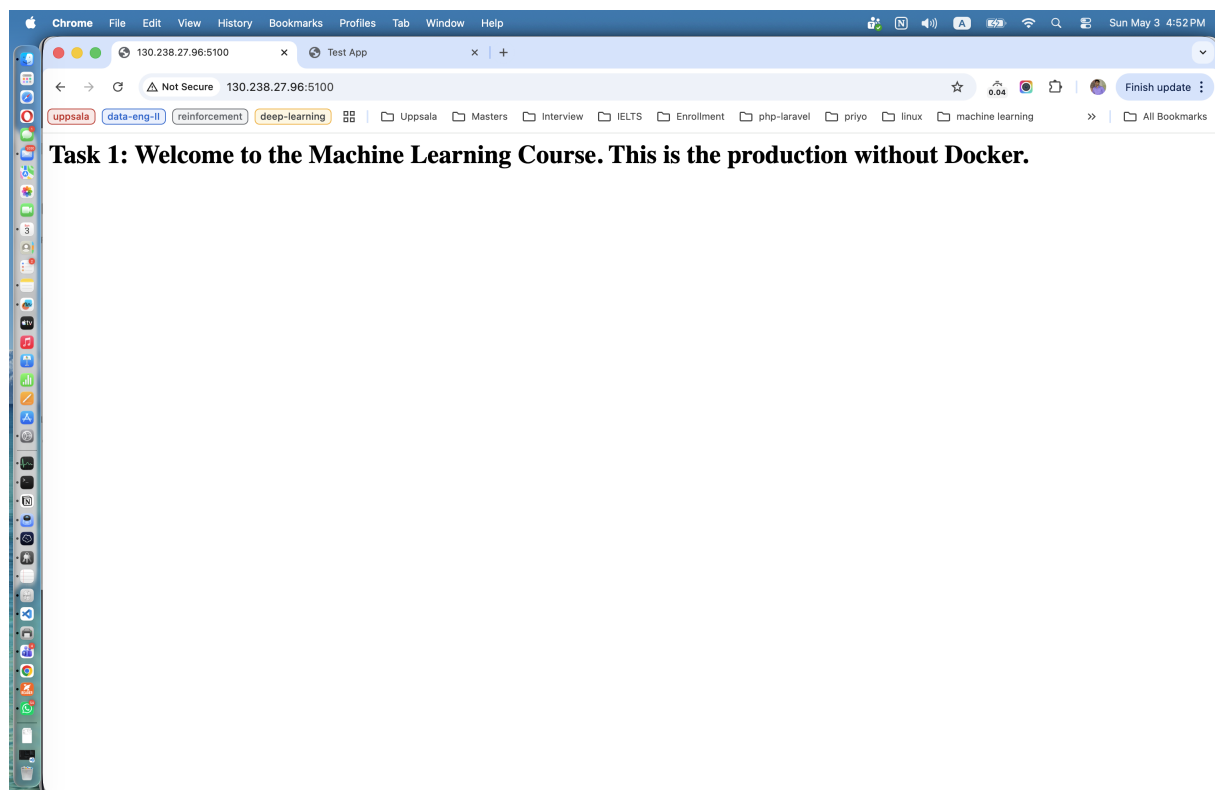


Figure 1: Welcome page of the Flask application running on the production server.

1.5 Questions

1. Explain how the application works.

The application has three tiers. A client sends input values to the Flask web page. Flask puts the prediction request into a RabbitMQ queue as a Celery task. A Celery worker picks up the task, loads the model from `model.h5` and `model.json`, runs the prediction, and saves the result. Flask then reads the result and shows it on the `/predictions` page. This design keeps the web server fast because heavy model computation runs in a separate worker process.

2. What are the drawbacks of the contextualization strategy adopted in Task 1?

1. **Slow deployment:** every new VM must download and install all packages from the internet at boot time. This takes 10–15 minutes.

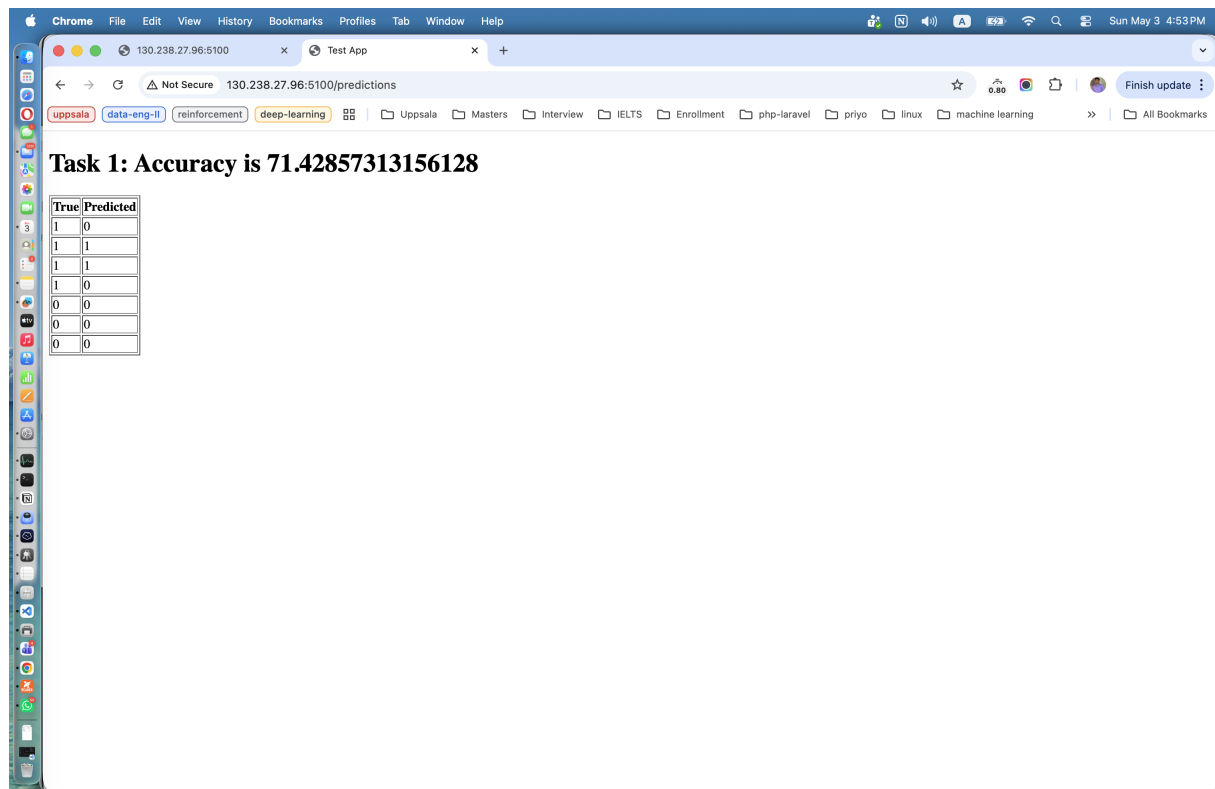


Figure 2: Predictions page showing diabetes prediction results returned by the Celery worker.

2. **No reproducibility:** the script runs `apt_upgrade: true`, so the latest package versions are installed. A new VM created later may get different package versions and behave differently.
3. **No isolation:** all services share the same OS. A package conflict or a crashed service can bring down the whole server.
4. **No automatic restart:** services are started as background processes. If a process crashes, nothing restarts it.
5. **Hard to scale:** adding a Celery worker requires logging in to the VM and running commands manually. There is no automated way to scale.
6. **No persistence:** if the VM is deleted, all data and configuration are lost. The whole setup must be done again from scratch.

3. How can we reduce the deployment time?

Most of the time is spent downloading and installing packages on each new VM. A pre-built Docker image solves this. All dependencies are installed once and saved to an image registry. When the VM boots, it only pulls the image and starts a container. This takes seconds instead of minutes. Another option is to create a custom OpenStack snapshot with all packages already installed. The VM then boots from that snapshot and skips the `apt install` step entirely.

4. Summary

In Task 1, I showed how CloudInit can fully automate the setup of a model serving application on a new VM. The `cloud-cfg.txt` file did everything: it updated the system, installed Python and RabbitMQ, cloned the code, and started all three services in the right order. I used the

OpenStack Python API to start the whole process from the client VM without needing to log in to the production server manually. The prediction service was accessible over HTTP after deployment. The main problems with this approach are slow boot time, no isolation, no automatic restart, and no easy scaling. Task 2 addresses these problems by using Docker containers.

2 Task 2: Single Server Deployment with Docker Containers

2.1 Overview

Task 2 repeats the deployment from Task 1 but uses Docker and Docker Compose. Each service runs in its own container. This gives isolation between services and makes it easy to add or remove Celery workers with one command.

2.2 Docker Compose Architecture

The `docker-compose.yml` defines three services:

Listing 3: `docker-compose.yml` for the production server

```
1 version: "3"
2 services:
3   web:
4     build:
5       context: .
6       network: host
7     restart: always
8     ports:
9       - "5100:5100"
10    depends_on:
11      - rabbit
12  rabbit:
13    hostname: rabbit
14    image: rabbitmq:3.11-management
15    environment:
16      - RABBITMQ_DEFAULT_USER=rabbitmq
17      - RABBITMQ_DEFAULT_PASS=rabbitmq
18    ports:
19      - "5672:5672"
20      - "15672:15672"
21  worker_1:
22    build:
23      context: .
24      network: host
25    hostname: worker_1
26    entrypoint: celery
27    command: -A workerA worker --loglevel=debug
28    links:
29      - rabbit
30    depends_on:
31      - rabbit
```

- **web**: the Flask frontend, built from the local `Dockerfile`. It listens on port 5100 and waits for RabbitMQ to start first.
- **rabbit**: the official RabbitMQ image with the management plugin. The login credentials are set through environment variables.

- **worker_1**: the Celery worker, built from the same Dockerfile but started with the Celery command instead of Flask.

2.3 CloudInit Configuration (with Docker)

The `cloud-cfg.txt` for this task installs Docker CE and starts the application with Docker Compose at boot time. Python packages are no longer installed directly on the VM:

Listing 4: cloud-cfg.txt – single server with Docker

```
1 #cloud-config
2 apt_update: true
3 apt_upgrade: true
4 packages:
5   - apt-transport-https
6   - ca-certificates
7   - curl
8   - software-properties-common
9 runcmd:
10  - curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -
11  - add-apt-repository -y "deb [arch=amd64] https://download.docker.com/linux/ubuntu
    focal stable"
12  - apt-get update -y
13  - apt-get install -y docker-ce
14  - git clone https://github.com/sztoor/model_serving.git
15  - docker compose -f /model_serving/single_server_with_docker/production_server/docker
    -compose.yml up -d
```

2.4 Scaling the Workers

Horizontal scaling was tested on the production VM after deployment.

Listing 5: Scaling Celery workers from 1 to 3

```
# Initial state -- one worker
sudo docker ps

# Scale up to 3 workers
sudo docker compose up --scale worker_1=3 -d

# Verify three workers are running
sudo docker ps

# Scale back down
sudo docker compose up --scale worker_1=1 -d
```

With three workers, five containers were running at the same time: `rabbit`, `web`, `worker_1_1`, `worker_1_2`, and `worker_1_3`. No changes were needed in Flask or RabbitMQ to add the extra workers.

2.5 Questions

1. What problems does Task 2 deployment solve compared to Task 1?

Docker fixes several problems from Task 1. Each service runs in its own container, so there are no dependency conflicts between Flask, Celery, and RabbitMQ. The `restart: always` setting means the web container restarts on its own if it crashes. Adding workers takes one command

```

Terminal
File Edit View Window Help
data-engineerin... data-engineering-II-VM-2
ubuntu@arnab-prod-server-with-docker-1548:~$ sudo docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
bd5899e104fb   production_server-worker_1          "celery -A workerA w..." 29 minutes ago Up 29 minutes
a99491d16964   production_server-web               "python ./app.py --h..." 29 minutes ago Up 29 minutes    0.0.0.0:5100->5100/tcp, [::]:5100->5100/tcp
497e17ff6763   rabbitmq:3.11-management           "docker-entrypoint.s..." 30 minutes ago Up 29 minutes    4369/tcp, 5671/tcp, 0.0.0.0:5672->5672/tcp, [::]:5672->5672/tcp, 15671/tcp, 1569
1-15692/tcp, 25672/tcp, 0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp
production_server-rabbit-1
ubuntu@arnab-prod-server-with-docker-1548:~$

```

Figure 3: `docker ps` output before scaling up the workers. The web service, RabbitMQ, and one worker are running.

```

Terminal
File Edit View Window Help
data-engineerin... data-engineering-II-VM-2
ubuntu@arnab-prod-server-with-docker-1548:/model_serving/single_server_with_docker/production_server$ sudo docker compose up --scale worker_1=3 -d
WARN[0000] /model_serving/single_server_with_docker/production_server/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential
confusion
[+] Running 5/5
  ✓ Container production_server-rabbit-1 Running
  ✓ Container production_server-worker_1-1 Running
  ✓ Container production_server-worker_1-2 Running
  ✓ Container production_server-worker_1-3 Started
  ✓ Container production_server-worker_1-2 Started
ubuntu@arnab-prod-server-with-docker-1548:/model_serving/single_server_with_docker/production_server$ sudo docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
73d5c7e29761   production_server-worker_1          "celery -A workerA w..." 13 seconds ago Up 10 seconds
879717f5b42e   production_server-worker_1          "celery -A workerA w..." 13 seconds ago Up 8 seconds
bd5899e104fb   production_server-worker_1          "celery -A workerA w..." 49 minutes ago Up 49 minutes
a99491d16964   production_server-web               "python ./app.py --h..." 49 minutes ago Up 49 minutes    0.0.0.0:5100->5100/tcp, [::]:5100->5100/tcp
497e17ff6763   rabbitmq:3.11-management           "docker-entrypoint.s..." 49 minutes ago Up 49 minutes    4369/tcp, 5671/tcp, 0.0.0.0:5672->5672/tcp, [::]:5672->5672/tcp, 15671/tcp, 1569
1-15692/tcp, 25672/tcp, 0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp
production_server-rabbit-1
ubuntu@arnab-prod-server-with-docker-1548:/model_serving/single_server_with_docker/production_server$

```

Figure 4: `docker ps` output after scaling up to three Celery workers. The web service, RabbitMQ, and three workers are running.

with `-scale`. The `Dockerfile` fixes the exact build environment, so the deployment is more reproducible than installing packages directly with `pip`.

2. What are the outstanding issues that Task 2 cannot address?

- **Single point of failure:** all containers run on one VM. If that VM goes down, the whole application stops.
- **No cross-node scaling:** Docker Compose cannot spread containers across several VMs. All workers share the resources of one machine.
- **Manual deployment:** a new version requires running `docker compose up` by hand. There is no automated pipeline.
- **No load balancing:** the web service has one container. High traffic cannot be spread across multiple web replicas automatically.

3. What are the possible solutions to address those outstanding issues?

A container orchestrator such as **Kubernetes** or **Docker Swarm** can run containers on many VMs, restart failed containers, and deploy updates without downtime. A CI/CD tool such as Git Hooks (Task 4) or GitHub Actions can rebuild and redeploy the application automatically when code changes. An ingress controller or load balancer placed in front of several web containers can spread traffic across all of them.

4. Horizontal vs. vertical scalability. Which does Task 2 use?

Vertical scalability means giving more CPU or RAM to one machine. **Horizontal scalability** means running more instances of a service. Task 2 uses **horizontal** scalability. The `-scale worker_1=3` command adds more worker containers instead of upgrading the VM. Because all containers are still on one host, this is intra-node horizontal scaling. Scaling across multiple hosts would need an orchestrator.

5. Summary

Task 2 improved on Task 1 by putting every service into a container. Docker Compose managed RabbitMQ, Flask, and Celery with clear startup order and automatic restart. Scaling workers became simple with the `-scale` flag. The CloudInit script became shorter: it now only installs Docker and runs one `docker compose up` command. After scaling to three workers, the predictions page was still working correctly. This confirmed that all workers shared the same RabbitMQ queue and processed tasks in parallel.

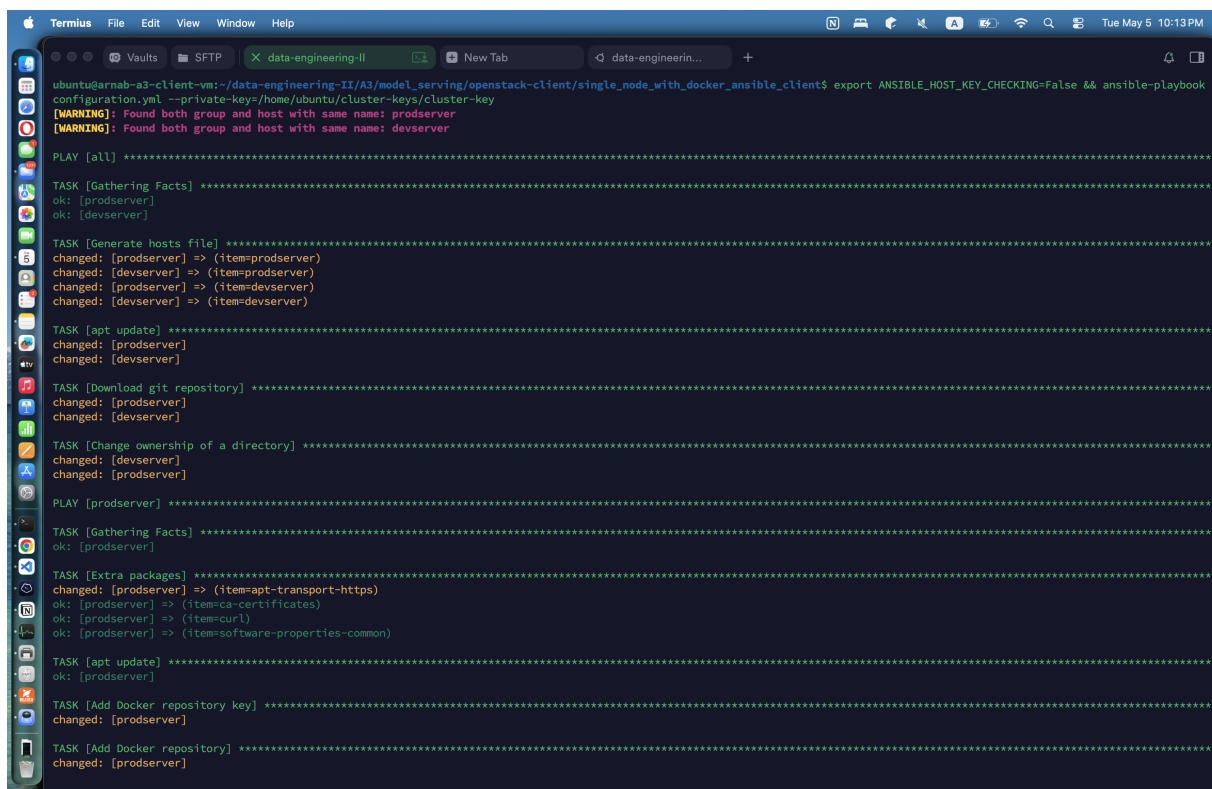
3 Task 3: Deployment of Multiple Servers Using Ansible

3.1 Overview

Task 3 adds a third VM to the setup. A client VM now manages two other VMs: a production server and a development server. The OpenStack Python API creates both VMs. An Ansible playbook then configures them at the same time. This replaces the CloudInit-only approach used in Tasks 1 and 2.

3.2 Infrastructure

- **Client VM:** runs the OpenStack client and Ansible. It creates the other two VMs and runs the playbook to configure them.
- **Production server:** runs the Dockerised application from Task 2.
- **Development server:** holds the development code and the model training script.



```

ubuntu@arnab-a3-client-vm:~/data-engineering-II/A3/model_serving/openstack-client/single_node_with_docker_ansible_client$ export ANSIBLE_HOST_KEY_CHECKING=False && ansible-playbook configuration.yml --private-key=/home/ubuntu/cluster-keys/cluster-key
[WARNING]: Found both group and host with same name: prodserver
[WARNING]: Found both group and host with same name: devserver

PLAY [all] *****
TASK [Gathering Facts] *****
ok: [prodserver]
ok: [devserver]

TASK [Generate hosts file] *****
changed: [prodserver] => (item=prodserver)
changed: [devserver] => (item=prodserver)
changed: [prodserver] => (item=devserver)
changed: [devserver] => (item=devserver)

TASK [apt update] *****
changed: [prodserver]
changed: [devserver]

TASK [Download git repository] *****
changed: [prodserver]
changed: [devserver]

TASK [Change ownership of a directory] *****
changed: [devserver]
changed: [prodserver]

PLAY [prodserver] *****
TASK [Gathering Facts] *****
ok: [prodserver]

TASK [Extra packages] *****
changed: [prodserver] => (item=apt-transport-https)
ok: [prodserver] => (item=ca-certificates)
ok: [prodserver] => (item=curl)
ok: [prodserver] => (item=software-properties-common)

TASK [apt update] *****
ok: [prodserver]

TASK [Add Docker repository key] *****
changed: [prodserver]

TASK [Add Docker repository] *****
changed: [prodserver]

```

Figure 5: Deployment architecture for Task 3: Client VM managing production and development servers using Ansible.

3.3 Questions

1. What is the difference between CloudInit and Ansible?

CloudInit runs once at first boot before SSH is available. It is meant for bootstrapping: creating users, adding SSH keys, and installing a small set of packages. It cannot be run again after the VM is created.

Ansible runs from a controller machine over SSH at any point in time. Playbooks are idempotent, meaning running them several times gives the same result. Ansible is built for configuration management and can apply changes to many machines at the same time.

In this task, CloudInit was used only to create `appuser` and add the SSH key. Ansible then connected over SSH and handled all further setup.

2. Explain the configurations in `dev-cloud-cfg.txt` and `prod-cloud-cfg.txt`.

Both files have the same structure. Each one creates one user called `appuser`. The user gets passwordless `sudo` rights, a `/bin/bash` shell, and the cluster SSH public key in `ssh_authorized_keys`. The `byobu_default: system` line turns on the Byobu terminal multiplexer for interactive sessions. The files are small by design because Ansible takes care of the rest.

3. What problem has Ansible solved?

Ansible solved the problem of configuring several machines at the same time. In Tasks 1 and 2, each VM needed its own CloudInit file and had to be set up separately. With Ansible, one playbook command from the client VM configures both the production server and the development server in one run. Common steps are listed once under `hosts: all`. Steps for only one server go in their own play. If the configuration needs to change, it is updated in one file and applied to all machines again.

4 Task 4: CI/CD Using Git Hooks

4.1 Overview

Task 4 sets up a continuous delivery pipeline with Git Hooks. When a new model is pushed to the bare Git repository on the production server, a `post-receive` hook copies the new model files to the live application folder. No manual file copying or service restart is needed.

4.2 Architecture

The pipeline uses the same three VMs from Task 3:

- **Development server:** trains a new model with `neural_net.py` and pushes the model files to the production server over Git.
- **Production server:** has a bare Git repository at `/home/appuser/my_project`. The `post-receive` hook copies new files to the live application folder on every push.
- **Client VM:** only needed for the initial SSH setup. It is not part of the push pipeline.

```
Dev Server --git push (SSH)--> Prod bare repo --post-receive hook-->
                                     /model_serving/ci_cd/production_server
```

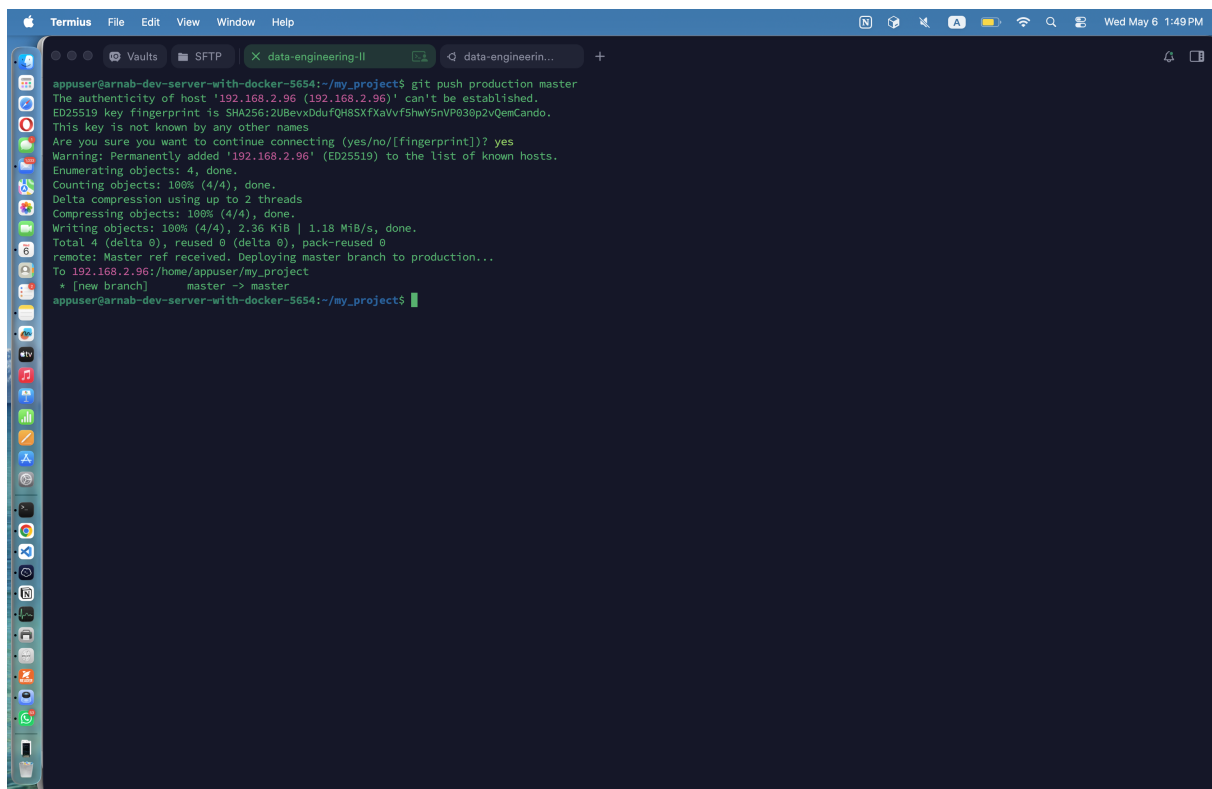


Figure 6: Deployment pipeline: development server pushes model files to production server via Git, triggering the `post-receive` hook for automated deployment.

4.3 The post-receive Hook

The hook file is at `/home/appuser/my_project/hooks/post-receive` on the production server:

Listing 6: `post-receive` Git hook on the production server

```
#!/bin/bash
```

```

while read oldrev newrev ref
do
  if [[ $ref =~ */master$ ]];
  then
    echo "Master ref received. Deploying master branch to production..."
    sudo git --work-tree=/model_serving/ci_cd/production_server \
      --git-dir=/home/appuser/my_project checkout -f
  else
    echo "Ref $ref successfully received. Doing nothing: only the master \
branch may be deployed on this server."
  fi
done

```

The hook reads each pushed branch name. If the branch is master, it runs `git checkout -f` with `-work-tree` to copy the pushed files into the live application folder. Old model files are replaced. If any other branch is pushed, the hook prints a message and does nothing.

4.4 Running the Pipeline

Listing 7: Initial push from the development server

```

# On the development server
mkdir /home/appuser/my_project && cd /home/appuser/my_project
git init
cp /model_serving/ci_cd/development_server/model* .
git add .
git commit -m "initial model"
git remote add production appuser@<PROD-IP>:/home/appuser/my_project
git push production master

```

Listing 8: Updating the model and pushing to production

```

# Train a new model on the development server
sudo python3 /model_serving/ci_cd/development_server/neural_net.py

# Copy new model files to the git working directory
cp /model_serving/ci_cd/development_server/model* /home/appuser/my_project/

# Commit and push
cd /home/appuser/my_project
git add .
git commit -m "updated model"
git push production master

```

The expected output from the push confirms the hook ran:

```

remote: Master ref received. Deploying master branch to production...
To 192.168.x.x:/home/appuser/my_project
   c20d19e..666ca0c  master -> master

```

4.5 Questions

1. What are Git hooks? Explain the post-receive script.

Git hooks are shell scripts that Git runs at certain points in the workflow. They are stored in the `hooks/` folder of a repository. Each hook has a fixed name that matches the event it listens to.

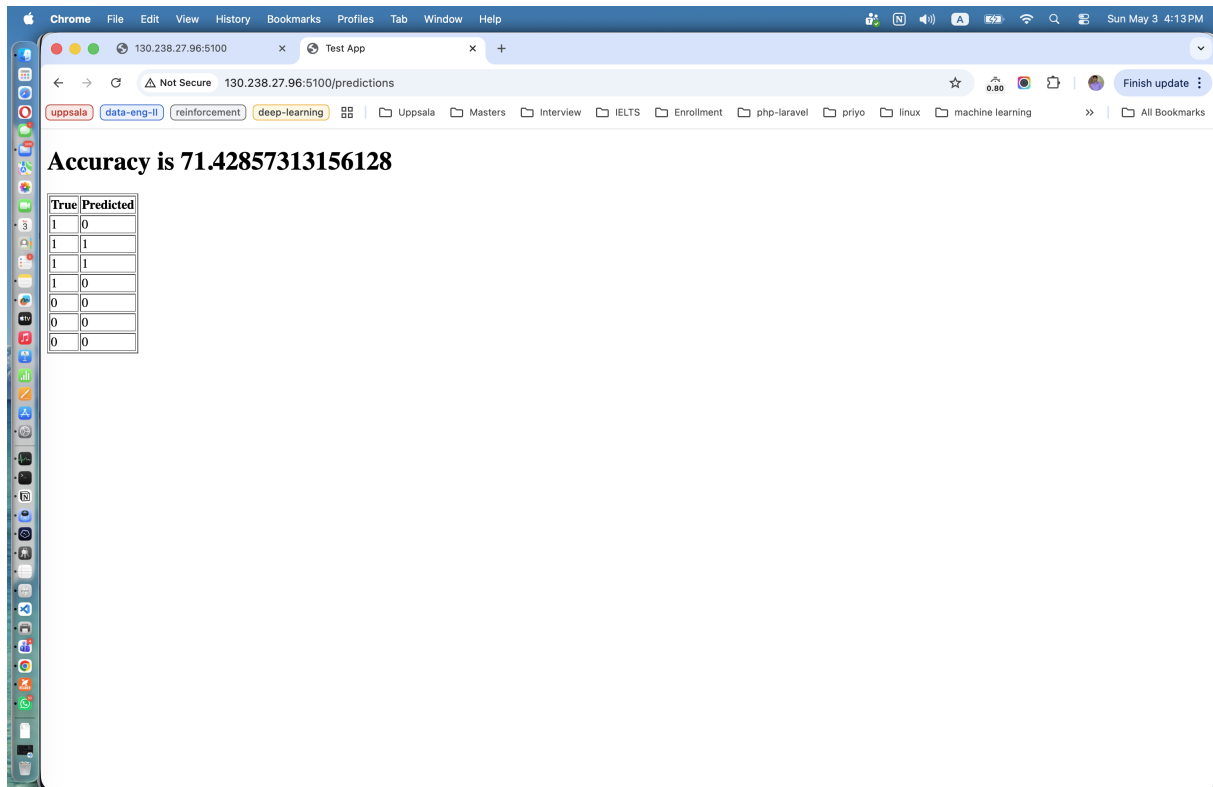


Figure 7: Output of `git push production master` showing the post-receive hook deploying the updated model.

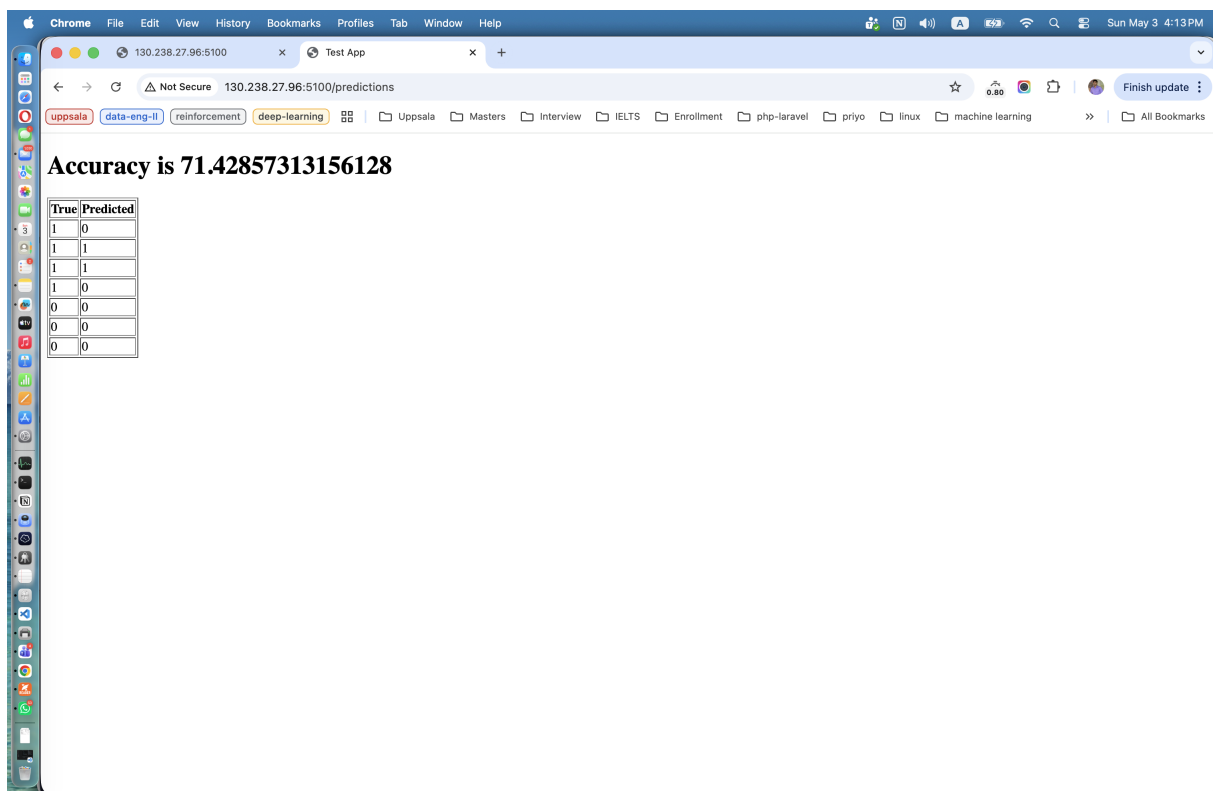


Figure 8: Predictions page reflecting the updated model after the CI/CD push.

The `post-receive` script in this task runs on the production server after a push is fully received. It reads each pushed branch. If the branch is `master`, it runs:

```
sudo git --work-tree=/model_serving/ci_cd/production_server \
  --git-dir=/home/appuser/my_project checkout -f
```

This copies the files from the push into the live folder, replacing the current model files. If any other branch is pushed, the hook only prints a message and stops. This makes sure only `master` commits reach production.

2. Why is the bare Git repository needed on the production server?

A bare repository only stores the Git object database. It has no working tree. It is used as the remote that the development server pushes to. A normal repository cannot accept a push to its active branch because Git does not know how to update the working tree safely. A bare repository has no active branch and no working files, so it always accepts pushes. The `post-receive` hook then copies files to a separate live folder using `-work-tree`. This keeps the Git storage and the running application files in separate locations.

3. Four Git hooks and their functionalities.

1. **pre-commit**: runs before a commit is saved. It is often used to run linters or tests. If it exits with an error, the commit is cancelled.
2. **commit-msg**: runs after a commit message is written. It can check that the message follows a required format, such as including a ticket number.
3. **pre-receive**: runs on the server before any branch is updated. It can reject a push based on rules such as file size or branch naming.
4. **post-receive**: runs on the server after all branches are updated. It is used to trigger deployments, send alerts, or update build status—as shown in this task.

4. Kubernetes deployment diagram for the multi-server setup.

The same application from Task 3 can be run in Kubernetes. A **Deployment** manages the Flask web pods. A **Service** of type `LoadBalancer` exposed the web pods to the outside. RabbitMQ runs in a **StatefulSet** behind a headless **Service**. The Celery workers run in a separate **Deployment** that can be scaled on its own. A **ConfigMap** stores non-secret settings such as the RabbitMQ hostname. A **Secret** stores credentials. Both the web pods and the worker pods mount a **PersistentVolumeClaim** to share the model files.

5. Summary

Task 4 completed the CI/CD pipeline that started in Task 3. A `post-receive` hook in the bare Git repository on the production server handles deployment automatically. The steps are simple: train a new model, copy the files to the local Git folder, commit, and push. The hook on the production server does the rest. This needs no extra tools beyond Git and SSH. It works well with the infrastructure set up by Ansible in Task 3. The result is a fast, repeatable path from model training to live deployment.

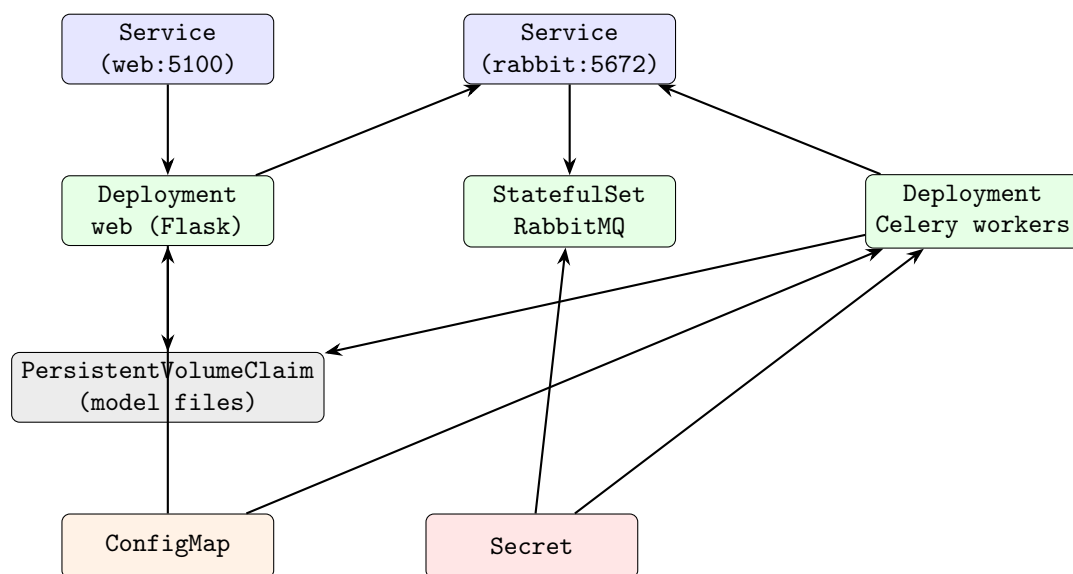


Figure 9: Kubernetes deployment diagram: Flask web Deployment behind a Service, RabbitMQ StatefulSet with headless Service, Celery worker Deployment, shared PersistentVolumeClaim for model files, ConfigMap and Secret for configuration and credentials.